COMP64401 Module 4: Logic Programming, Rules and Datalog

Structured Study Notes

Prepared from uploaded lecture transcripts and slides

2026-05-16

Contents

Topic and scope

This lecture block introduces Datalog as a logic-programming language for knowledge representation, reasoning, and deductive databases. It covers Datalog syntax and semantics, Herbrand models, fixed-point reasoning, reasoning tasks, limitations and extensions, comparisons with description logics, and applications.

Course context. COMP64401: *Logics for Knowledge Representation and Reasoning*. This module follows propositional logic and description logic, then uses Datalog to show another rule-based, database-oriented logic paradigm.

Document sources. Uploaded transcripts for videos 4.1, 4.3, 4.4, and 4.5, plus the uploaded Week 7+8 slide PDF.

Notation. `[UNCLEAR]` marks sections where the transcript is garbled, the slide and spoken explanation may disagree, or the lecture likely contains a typo.

1. Recap: where Datalog fits after propositional and description logic

Before Datalog, the course covered two logics.

1.1 Propositional logic

The course has already covered:

- Syntax: propositional variables and formulae.
- Semantics: valuations.
- Reasoning tasks: satisfiability, validity, implication.
- Algorithms: tableau-style reasoning, involving normalising input and saturating a set using rules.

1.2 Description logic

The course has already covered:

- Syntax: classes, axioms, ABoxes, TBoxes, knowledge bases.
- Semantics: interpretations satisfying axioms; entailment.
- Reasoning tasks: entailment testing, instance retrieval, classification.
- Algorithms: consequence-based reasoning, again involving normalisation and saturation.
- Usage in knowledge representation and reasoning.

The lecturer explicitly connects this to Datalog: Datalog will also have syntax, semantics, reasoning tasks, and algorithmic reasoning procedures.

2. Logic programming and Datalog: basic idea

2.1 Logic programming

Intuition. Logic programming is a rule-based logic paradigm. Instead of writing arbitrary formulae, we write rules saying when certain atoms follow from other atoms.

The lecture lists logic programming as a paradigm used for:

- knowledge representation and reasoning;
- deductive databases;
- systems based on atoms, rules, and clauses.

Variants mentioned:

- Prolog;
- Answer Set Programming;
- Constraint Logic Programming;
- Datalog.

This course focuses on Datalog, but later discusses limitations and extensions.

2.2 Running toy example: weather and cold legs

The slides give a Datalog-style program:

```
WeatherC(x) :- Temp(x).
WeatherC(x) :- Preciptn(x).
Temp(x) :- Cold(x).
Cold(x) :- BelowF(x).
Preciptn(x) :- Rain(x).
Preciptn(x) :- Snow(x).
Slippery(x) :- Rain(x), BelowF(x).

ColdLegs(x) :- Person(x), wears(x,y), Shorts(y),
               isLocIn(x,z), hasWeather(z,w), Cold(w).

Person(Bijan).
wears(Bijan, S1).
Shorts(S1).
isLocIn(Bijan,M).
hasWeather(M,W).
BelowF(W).
```

Intuitively:

- something is a weather condition if it is a temperature or precipitation;
- below-freezing things are cold;
- rain and below-freezing conditions imply slippery conditions;
- a person has cold legs if they wear shorts, are located somewhere, that location has some weather, and that weather is cold;
- the ground facts say Bijan wears shorts and is in a location whose weather is below freezing.

The lecturer points out the analogy with description logic ABoxes and TBoxes: the bottom part gives facts about named individuals, while the top part gives general rules. In Datalog, the facts are called **ground facts** or **ground atoms**, not ABox assertions.

3. Datalog syntax

3.1 Vocabulary: variables, individuals, predicates

Key concept: terms

Intuition. Terms are the things that can appear as arguments of predicates. In this version of Datalog, terms are either variables or named individuals/constants.

Formal definition. Let N_V, N_I, and N_P be pairwise disjoint sets of variables, individuals, and predicates. Each predicate $P \in N_P$ has an arity $n \in \mathbb{N}$. The set of terms is:

$$N_{VI} := N_V \cup N_I.$$

Individuals are also often called **constants**.

Key concept: atom

Intuition. An atom is a predicate applied to the correct number of arguments.

Formal definition. If $P \in N_P$ has arity n, and each $a_i \in N_{VI}$, then:

$$P(a_1, \ldots, a_n)$$

is an atom.

Examples:

```
Person(Bijan)
Rain(x)
wears(x,y)
hasWeather(z,w)
```

The same variable may occur multiple times; the syntax does not forbid this.

3.2 Rules

Key concept: Datalog rule

Intuition. A rule says: if all atoms in the body hold, then the atom in the head holds.

Formal definition. A rule is an expression:

$$B \ :- \ A_1, \ldots, A_m$$

where B and all A_i are atoms, $m \geq 0$, and **all variables in the head B must also occur in some body atom A_i**. A Datalog program is a finite set of rules.

Terminology:

- B is the **head** of the rule.
- $A_1, \ldots, A_m$ is the **body** of the rule.
- `:-` is read as "if" or as a leftward implication: body implies head.

Example:

```
Slippery(x) :- Rain(x), BelowF(x).
```

Read:

$$\text{If } \mathit{Rain}(x) \text{ and } \mathit{BelowF}(x), \text{ then } \mathit{Slippery}(x).$$

Formally:

$$\mathit{Rain}(x) \wedge \mathit{BelowF}(x) \Rightarrow \mathit{Slippery}(x).$$

Or equivalently:

$$\mathit{Slippery}(x) \Leftarrow \mathit{Rain}(x) \wedge \mathit{BelowF}(x).$$

3.3 Facts

Key concept: ground fact

Intuition. A fact is a rule with no body. It is asserted unconditionally.

Example:

```
Person(Bijan).
```

This is shorthand for:

```
Person(Bijan) :-
```

That is, the body is empty. The lecturer stresses this because it matters later for the Herbrand model and fixed-point computation.

Ground atom

A **ground atom** is an atom with no variables, only individuals/constants.

Examples:

```
Person(Bijan)
wears(Bijan,S1)
BelowF(W)
```

Non-ground atoms contain variables, for example:

```
Person(x)
wears(x,y)
```

3.4 Safety condition: no fresh variables in the head

The rule:

```
hasParent(x,y) :- Person(x).
```

is **not** a valid Datalog rule because y occurs in the head but not in the body. This would require inventing an unnamed parent y, which plain Datalog does not allow.

This condition is crucial later: it is one reason Datalog can restrict reasoning to the **active domain**, meaning the named individuals already present in the program.

3.5 Rules as Horn clauses

Datalog rules can be understood as Horn clauses.

Example:

```
Slippery(x) :- Rain(x), BelowF(x).
```

This abbreviates:

$$Slippery(x) \Leftarrow Rain(x) \wedge BelowF(x).$$

Using implication-as-disjunction:

$$\varphi \Leftarrow \psi \quad \text{means} \quad \varphi \vee \neg\psi.$$

So:

$$Slippery(x) \vee \neg(Rain(x) \wedge BelowF(x)).$$

By De Morgan's laws:

$$Slippery(x) \vee \neg Rain(x) \vee \neg BelowF(x).$$

This has exactly one positive literal, $Slippery(x)$. Clauses with at most one positive literal are **Horn clauses**.

Key intuition. Datalog rules are computationally convenient versions of a restricted logical form: Horn clauses. This Horn restriction is a major reason Datalog reasoning is tractable under the bounded-arity condition.

4. Datalog semantics

The lecture discusses three equivalent semantics for Datalog:

1. model-theoretic semantics;
2. minimal-model-theoretic semantics;
3. fixed-point semantics.

The slides state that they are equivalent, and the lecture develops them in that order: model-theoretic semantics gives the meaning, minimal-model semantics gives a canonical model, and fixed-point semantics gives an algorithm.

4.1 Model-theoretic semantics

Key concept: interpretation

Intuition. An interpretation gives meaning to predicates and individuals.

Formal definition. For a Datalog program P, an interpretation is:

$$\mathcal{J} = (\Delta^{\mathcal{J}}, \cdot^{\mathcal{J}})$$

where:

- $\Delta^{\mathcal{J}}$ is a non-empty interpretation domain;
- each predicate A of arity n is mapped to an n-ary relation:

$$A^{\mathcal{J}} \subseteq (\Delta^{\mathcal{J}})^n;$$

- each individual b is mapped to an element:

$$b^{\mathcal{J}} \in \Delta^{\mathcal{J}}.$$

The main difference from description logic semantics is that Datalog has explicit variables, so the semantics also needs substitutions.

Key concept: substitution

Intuition. A substitution assigns domain elements to variables.

Formal definition.

$$\sigma : N_V \to \Delta^{\mathcal{J}}.$$

For a term d, its value under interpretation $\mathcal{J}$ and substitution σ is:

$$d^{\mathcal{J},\sigma} = \begin{cases} \sigma(d), & \text{if } d \in N_V, \\ d^{\mathcal{J}}, & \text{if } d \in N_I. \end{cases}$$

So variables are interpreted using σ, and individuals/constants are interpreted using $\mathcal{J}$.

Satisfaction of atoms

For an atom:

$$P(d_1, \dots, d_n),$$

$\mathcal{J}, \sigma$ satisfies the atom if the interpreted tuple belongs to the predicate relation:

$$(d_1^{\mathcal{J},\sigma}, \ldots, d_n^{\mathcal{J},\sigma}) \in P^{\mathcal{J}}.$$

[UNCLEAR/OCR] The slide parse shows a subset symbol in this line, but the transcript says the tuple "belongs to" the predicate interpretation, so the intended symbol is membership $\in$.

Satisfaction of rules

For a rule:

$$H :- A_1, \ldots, A_m,$$

$\mathcal{J}, \sigma$ satisfies the rule if:

$$\text{if } \mathcal{J}, \sigma \models A_i \text{ for every } i, \text{ then } \mathcal{J}, \sigma \models H.$$

In words: if the body is true, the head must be true.

Satisfaction of programs

$\mathcal{J}, \sigma$ satisfies a program P if it satisfies every rule in P.

Entailment

A Datalog program P entails an atom α, written:

$$P \models \alpha,$$

if for every interpretation $\mathcal{J}$ and every substitution σ, whenever $\mathcal{J}, \sigma$ satisfies P, it also satisfies α.

Universal quantification of variables

Variables in rules are universally quantified. For example:

```
Slippery(x) :- Rain(x), BelowF(x).
```

corresponds to:

$$\forall x \, (Rain(x) \wedge BelowF(x) \Rightarrow Slippery(x)).$$

Equivalently, as a Horn clause:

$$\forall x \, (Slippery(x) \vee \neg Rain(x) \vee \neg BelowF(x)).$$

The lecturer explicitly contrasts this with existential reading: the rule is true for all x, not for some x.

5. Herbrand base and Herbrand model

The model-theoretic definition quantifies over all interpretations and substitutions, which is not directly usable as an algorithm because there are infinitely many interpretations. The lecture introduces Herbrand-style semantics to avoid searching through all models.

5.1 Herbrand base

Key concept: Herbrand base

Intuition. The Herbrand base is the set of all possible ground atoms that can be formed using the predicates and named individuals occurring in the program.

Formal definition. For a Datalog program P:

$$HB(P) := \{A(b_1, \dots, b_n) \mid b_i \in N_I,\ A \in N_P \text{ occurs in } P,\ A \text{ has arity } n\}.$$

So $HB(P)$ contains no variables. It is all possible ground combinations of the program's predicates with the program's individuals.

Example

If P contains individuals B and U, and predicates such as `Person`, `Slippery`, `Rain`, and `wears`, then the Herbrand base includes atoms such as:

```
Person(B)
Person(U)
Slippery(B)
Slippery(U)
Rain(B)
Rain(U)
wears(B,B)
wears(B,U)
wears(U,B)
wears(U,U)
...
```

The lecturer stresses that the Herbrand base itself is not the set of entailed facts. It is merely the set of all candidate ground atoms.

5.2 Herbrand model

Key concept: Herbrand model

Intuition. The Herbrand model is the smallest subset of the Herbrand base that contains all facts and is closed under the program's rules.

Formal definition.

$$HM(P) := \min_{\subseteq} \left\{ X \subseteq HB(P) \;\middle|\; \begin{array}{l} \text{if } H :- A_1, \dots, A_n \in P \\ \text{and all } \sigma(A_i) \in X, \\ \text{then } \sigma(H) \in X \end{array} \right\}.$$

Here σ maps variables to individuals in the program. The condition says: whenever the grounded body atoms are already in X, the grounded head must also be in X. The minimality condition says X contains nothing unnecessary.

Core theorem: entailment via the Herbrand model

For a Datalog program P and a ground atom α:

$$P \models \alpha \quad \text{iff} \quad \alpha \in HM(P).$$

This is one of the main results of the lecture. It means that instead of checking all models, we only need to inspect a single canonical model.

Active domain

The Herbrand model only talks about named individuals in the program. This is called the **active domain**.

This is a major difference from description logic: in description logic, we can often talk about anonymous individuals, such as "every person has a parent." In plain Datalog, we cannot introduce unnamed individuals in this way.

5.3 Size and complexity of the Herbrand base/model

Lemma

Let P be a Datalog program with fixed maximum predicate arity. Then:

1. $HB(P)$ is finite and polynomial in the size of P;
2. $HM(P)$ is finite and polynomial in the size of P.

Proof sketch from the lecture

Let:

- n be the maximum arity of predicates in P;
- p be the number of predicates in P;
- m be the number of individuals in P.

Then there are at most:

$$p \cdot m^n$$

ground atoms over P. Since $HM(P) \subseteq HB(P)$, the Herbrand model is also finite and polynomial in P, **provided that n is fixed**.

Exam flag

The lecturer explicitly emphasises that bounded predicate arity is critical. If arity is unbounded, n can grow with the input, and $p \cdot m^n$ can become exponential.

6. Worked example: computing a Herbrand model

The slides give a program using weather, clothes, and cold legs:

```
WeatherC(x) :- Temp(x).
Temp(x) :- Cold(x).
Cold(x) :- BelowF(x).
IoC(x) :- Shorts(x).
Person(x) :- wears(x,y), IoC(y).

ColdLegs(x) :- Person(x), wears(x,y), Shorts(y),
               isLocIn(x,z), hasWeather(z,w), Cold(w).

wears(B,S).
Shorts(S).
isLocIn(B,M).
hasWeather(M,W).
BelowF(W).
```

The Herbrand base contains all possible ground atoms over the individuals B, S, M, W and the predicates in the program. For example:

```
WeatherC(B), WeatherC(S), WeatherC(M), WeatherC(W),
Temp(B), Temp(S), Temp(M), Temp(W),
Cold(B), Cold(S), Cold(M), Cold(W),
wears(B,B), wears(B,M), wears(B,S), wears(B,W),
...
```

The slides list the Herbrand model as containing:

```
wears(B,S)
Shorts(S)
isLocIn(B,M)
hasWeather(M,W)
BelowF(W)
IoC(S)
Person(B)
Cold(W)
ColdLegs(B)
```

Derivation steps shown or implied:

1. The ground facts are included immediately:
   ```
   wears(B,S)
   Shorts(S)
   isLocIn(B,M)
   hasWeather(M,W)
   BelowF(W)
   ```
2. From:
   ```
   IoC(x) :- Shorts(x)
   Shorts(S)
   ```
   infer:
   ```
   IoC(S)
   ```
3. From:
   ```
   Person(x) :- wears(x,y), IoC(y)
   wears(B,S)
   IoC(S)
   ```
   infer:
   ```
   Person(B)
   ```
4. From:
   ```
   Cold(x) :- BelowF(x)
   BelowF(W)
   ```
   infer:
   ```
   Cold(W)
   ```
5. From:
   ```
   ColdLegs(x) :- Person(x), wears(x,y), Shorts(y),
                  isLocIn(x,z), hasWeather(z,w), Cold(w)
   ```
   using the substitution:

   $$x \mapsto B, \quad y \mapsto S, \quad z \mapsto M, \quad w \mapsto W,$$

   and the known facts:
   ```
   Person(B)
   wears(B,S)
   Shorts(S)
   isLocIn(B,M)
   hasWeather(M,W)
   Cold(W)
   ```
   infer:
   ```
   ColdLegs(B)
   ```

Thus the lecture concludes, for example:

$$P \models Person(B)$$

and:

$$P \models ColdLegs(B).$$

[UNCLEAR] The displayed program also contains `Temp(x) :- Cold(x)` and `WeatherC(x) :- Temp(x)`, so from `Cold(W)` one would expect `Temp(W)` and then `WeatherC(W)`. The slide/transcript Herbrand model list stops at `Cold(W)` and `ColdLegs(B)`. This is worth checking in the recording or slides.

7. Fixed-point semantics and reasoning algorithm

The previous section defines $HM(P)$, but not an efficient way to compute it. The lecture rejects "guess a subset $X \subseteq HB(P)$ and check minimality" as bad, then introduces an iterative fixed-point method.

7.1 Immediate consequence operator

Key concept: immediate consequence operator

Intuition. Given a current set X of known ground atoms, the immediate consequence operator adds every rule head that can be derived in one rule application.

Formal definition. For a Datalog program P:

$$ICO_P(X) := X \cup \{\sigma(H) \in HB(P) \mid H :- A_1, \ldots, A_n \in P \text{ and all } \sigma(A_i) \in X\}.$$

So $ICO_P(X)$ contains everything already in X, plus all immediate consequences of rules whose grounded bodies are already in X.

Monotonicity of the operator

$$X \subseteq ICO_P(X).$$

The operator only adds atoms; it never removes them.

Kleene fixed-point theorem

The lecture invokes Kleene's fixed-point theorem: for a monotone operator on sets, the least fixed point exists and can be computed iteratively:

$$\bigcup_i ICO_P^i(\emptyset).$$

This least fixed point is usually denoted:

$$ICO_P^*(\emptyset).$$

It is finite and polynomial in size because:

$$ICO_P^*(\emptyset) \subseteq HB(P),$$

and $HB(P)$ is polynomial when predicate arity is bounded.

7.2 Fixed-point theorem for Datalog

Theorem

$$HM(P) = ICO_P^*(\emptyset).$$

This gives an algorithmic way to compute the Herbrand model.

Corollary

For a Datalog program P and a ground atom α:

$$P \models \alpha \quad \text{iff} \quad \alpha \in ICO_P^*(\emptyset).$$

So all ground atomic entailments can be computed by iterative rule application.

7.3 Naive algorithm for computing all ground atomic entailments

Input: Datalog program P.
Output: $HM(P)$.

Clean version of the lecture algorithm:

```
X := empty set
I := set of individuals in P

repeat
    X' := X

    for each rule H :- A1, ..., An in P do
        V := variables occurring in the rule

        for each substitution sigma : V -> I do
            if sigma(Ai) in X for every i = 1,...,n then
                add sigma(H) to X'

    if X' = X then
        return X

    X := X'
```

The important parts:

- X is the current set of derived ground atoms.
- X' is the next stage.
- Each iteration checks all rules and all substitutions.
- A fixed point is reached when $X' = X$, meaning no new atoms were added.

7.4 Optimisations

The lecture gives two immediate optimisations.

Optimisation 1: start with facts

Lemma:

$$ICO_P(\emptyset)$$

is the set of all facts in P.

So instead of starting with $X := \emptyset$, start with:

$$X := \{\text{facts in } P\}.$$

This avoids rediscovering all facts through empty-body rules.

Optimisation 2: focused search for substitutions

Rather than blindly trying every substitution $\sigma : V \to I$, focus only on substitutions that have a chance of succeeding: those for which the grounded body atoms are already in X.

The lecturer also notes that rule ordering and body-literal ordering can help. For example, if no atom matching `BelowF(...)` has been derived yet, there is no point considering rules requiring `BelowF` in the body.

Exam flag

The naive fixed-point method is conceptually important, but practical Datalog reasoners require optimisations. The lecturer explicitly says computing the whole Herbrand model to answer one query can be wasteful.

8. Datalog reasoning tasks

The lecture distinguishes several reasoning tasks.

8.1 Compute all ground atomic entailments

Task:

Given P, compute all ground atoms α such that:

$$P \models \alpha.$$

Solution:

Compute:

$$HM(P)$$

or equivalently:

$$ICO_P^*(\emptyset).$$

Every atom in the resulting Herbrand model is a ground atomic entailment.

8.2 Test a specific ground entailment

Task:

Given P and a ground atom α, decide whether:

$$P \models \alpha.$$

Naive solution:

1. Compute $HM(P)$.
2. Check whether:

$$\alpha \in HM(P).$$

The lecturer points out that this is often wasteful because it computes all entailments just to answer one yes/no question. A more goal-directed reasoner can reason backwards from α.

8.3 Answer conjunctive queries

Key concept: conjunctive query

Intuition. A conjunctive query asks for all tuples of individuals that make several atoms true simultaneously.

A query has the form:

$$q(\vec{x}) :- A_1, \dots, A_n.$$

The goal is to return all tuples $\vec{a}$ such that each query atom is entailed after substituting $\vec{a}$ for $\vec{x}$.

Formal task

Return all tuples $\vec{a}$ such that:

$$P \models A_1[\vec{x}/\vec{a}], \dots, P \models A_n[\vec{x}/\vec{a}].$$

Here $A_i[\vec{x}/\vec{a}]$ is the ground atom obtained from A_i by replacing each query variable x_j with the corresponding individual a_j.

Naive solution

1. Compute $HM(P)$.
2. For every vector $\vec{a}$ over individuals in P, check whether:

$$A_i[\vec{x}/\vec{a}] \in HM(P) \quad \text{for every } i.$$

3. If yes, output $\vec{a}$.

Again, this is conceptually easy but needs optimisations in practice.

8.4 Rule entailment

Key concept: rule entailment

Intuition. Instead of asking whether a ground atom follows, ask whether an entire rule follows from a program. This is a conceptual-level reasoning task, analogous to subsumption entailment in description logic.

Task:

Given a Datalog program P and a rule:

$$H :- A_1, \ldots, A_n,$$

decide whether:

$$P \models H :- A_1, \ldots, A_n.$$

Formal definition

P entails the rule $H :- A_1, \ldots, A_n$ if, for every interpretation $\mathcal{I}$ and every substitution/valuation σ, whenever $\mathcal{I}, \sigma$ satisfies every rule in P, it also satisfies the rule $H :- A_1, \ldots, A_n$.

Fresh-constant trick

The lecture gives a neat reduction to Herbrand-model computation.

Pick fresh constants:

$$\vec{c} = c_1, \ldots, c_k$$

one for each variable in the rule, with none of the c_i already occurring in P.

Add the grounded body atoms to the program:

$$P \cup \{A_1(\vec{c}), \ldots, A_n(\vec{c})\}.$$

Then check whether the grounded head is in the Herbrand model:

$$H(\vec{c}) \in HM(P \cup \{A_1(\vec{c}), \ldots, A_n(\vec{c})\}).$$

The lemma states:

$$P \models H :- A_1, \ldots, A_n$$

iff:

$$H(\vec{c}) \in HM(P \cup \{A_1(\vec{c}), \ldots, A_n(\vec{c})\}).$$

This is the first non-ground entailment task in the lecture, and the trick reduces it back to ground reasoning.

9. What Datalog can express well

The lecturer first discusses “non-limitations”: things Datalog is good at expressing.

9.1 Hierarchies

Datalog easily expresses class/property hierarchies.

Examples:

```
Animal(x) :- Bird(x).
Bird(x) :- Duck(x).
```

Read:

- every bird is an animal;
- every duck is a bird.

[UNCLEAR] The transcript briefly says “dog” in this hierarchy example, but the slide says `Duck(x)`, which matches the intended hierarchy.

9.2 Domain and range of relations

Example statement:

> Ownership happens between a person and an inanimate object.

Datalog rules:

```
Person(x) :- owns(x,y).
InObj(y) :- owns(x,y).
```

So if `owns(x,y)` holds, infer that x is a person and y is an inanimate object.

9.3 Types in general, including higher-arity predicates

Datalog predicates can have arity greater than two, so it can express types for ternary relations.

Example:

```
Product(x)   :- Sale(x,y,z).
Assistant(y) :- Sale(x,y,z).
Shop(z)      :- Sale(x,y,z).
```

This says a sale relation connects a product, a sales assistant, and a shop. This is a key advantage over description logics such as $\mathcal{EL}$, which use unary and binary predicates only.

9.4 Complex relational structures

Datalog can describe non-tree-shaped relational structures.

Example statement:

> Two wheels connected to a frame make a bicycle.

Rule:

```
Bicycle(x) :- hasPart(x,y), hasPart(x,z), hasPart(x,f),
              Wheel(y), Wheel(z), Frame(f),
              connectedTo(y,f), connectedTo(z,f).
```

The important point is that the variables y, z, f can be connected in an arbitrary graph-like pattern. The slide includes a bicycle sketch illustrating this non-tree-shaped relational structure.

9.5 Implications between predicates/properties

Datalog can express inverse-like and subproperty-like relationships.

Examples:

```
isParentOf(x,y) :- hasParent(y,x).
isPartOf(x,y)   :- hasPart(y,x).
hasChild(x,y)   :- hasDaughter(x,y).
```

The first says `isParentOf` is the inverse of `hasParent`. The second says `isPartOf` is the inverse of `hasPart`. The third says having a daughter implies having a child.

[UNCLEAR / lecturer-noted typo] The slide/transcript contains a variable-order issue for `hasDaughter`/`hasChild`. The lecturer explicitly says the variables should not be swapped: both should be `x,y`, not one occurrence as `y,x`.

9.6 Recursion

Datalog supports recursive rules.

Example: define `isRelatedTo` as the transitive symmetric closure of `hasParent`.

```
isRelatedTo(x,y) :- hasParent(x,y).
isRelatedTo(x,y) :- isRelatedTo(y,x).
isRelatedTo(x,y) :- isRelatedTo(x,z), isRelatedTo(z,y).
```

Intuition:

1. Parent links imply relatedness.
2. Relatedness is symmetric.
3. Relatedness is transitive.

The lecturer calls this a powerful feature: with these three rules, Datalog captures finite parent-path relatedness in either direction, while still keeping polynomial-time reasoning under bounded arity.

10. Limitations of plain Datalog

10.1 No anonymous individuals

Plain Datalog cannot express statements that require unnamed/existential individuals.

Cannot express:

> Persons have parents.

The tempting invalid rule would be:

```
hasParent(x,y) :- Person(x).
```

But y occurs only in the head, so it is a fresh head variable. Datalog forbids this.

Cannot express:

> Each bicycle has two wheels and a frame.

That would require generating unnamed wheel/frame individuals from the fact that something is a bicycle.

Datalog also disallows function terms, so this is invalid:

```
hasParent(x, motherOf(x)) :- Person(x).
```

The lecturer connects this limitation directly to the small-model property: because Datalog never creates new individuals, reasoning can stay within the named individuals of the program.

10.2 No disjunction in rule heads

Plain Datalog rule heads contain a single atom.

Cannot express:

> Students are undergraduate students or postgraduate students.

Invalid form:

```
UGStudent(x) v PGTStudent(x) :- Student(x).
```

This would require a disjunctive head.

10.3 No negation

Plain Datalog has no full negation.

It cannot express:

- disjointness, for example persons and inanimate objects do not overlap;
- negation in the body;
- negation in the head.

Example of a rule not available in plain Datalog:

```
UGStudent(x) :- Student(x), not PGTStudent(x).
```

The lack of negation is connected to the lack of disjunction: with full negation and conjunction, one can recover disjunction via De Morgan dualities, so allowing negation would fundamentally change the logic.

11. Extensions of Datalog

11.1 Datalog with negation

A common extension allows negation in rule bodies.

Example:

```
UGStudent(x) :- Student(x), not PGTStudent(x).
Student(Bob).
```

Under a negation-as-failure/default reading:

- if `Student(Bob)` is known;
- and `PGTStudent(Bob)` is not entailed;
- infer `UGStudent(Bob)`.

This is non-monotonic: if we later add:

```
PGTStudent(Bob).
```

then we must withdraw the previous conclusion that Bob is an undergraduate student.

Key concept: monotonicity

Formal definition. Let $\mathcal{L}$ be a logic with entailment relation $\models$. $\mathcal{L}$ is monotonic if, for all sets of formulae P, P' and every axiom α:

$$\text{if } P \models \alpha, \text{ then } P \cup P' \models \alpha.$$

If this property fails, the logic is non-monotonic.

Intuition. In a monotonic logic, adding information cannot invalidate old entailments. In a non-monotonic logic, adding information may force us to withdraw conclusions.

The lecturer mentions forms of non-monotonicity:

- negation as failure;
- circumscription;
- default reasoning.

The undergraduate-student example is a default rule: by default, a student is treated as an undergraduate unless there is evidence they are postgraduate.

11.2 Semantics for negation

The lecture mentions several semantics for Datalog with negation:

- negation as failure;
- stable model semantics;
- Answer Set Programming;
- stratified negation.

Negation is easier when there are no cyclic dependencies through negation. This is the stratified case.

With cyclic dependencies involving negation, things become harder:

- the unique/minimal Herbrand model may be lost;
- reasoning may require considering many Herbrand models;
- different entailment notions may arise.

[UNCLEAR] The transcript says "cautious entailment and brief entailment"; this is likely a garbling of a technical term. Listen back to confirm the exact term.

11.3 Disjunctive Datalog

Disjunctive Datalog allows disjunction in rule heads.

Example:

```
UGStudent(x) v PGTStudent(x) :- Student(x).
```

The lecture mentions disjunctive stable model semantics and the DLV reasoner.

[UNCLEAR] The transcript's reasoner name is garbled, but the slide says DLV.

11.4 Datalog with data types

Datalog is extended in applications with data types such as:

- primitive types: numbers, strings;
- composite/user-defined types: records of integers;
- operations and aggregations;
- comparisons such as $\leq$, $=$;
- arrays, lists, records;
- pointers.

The lecturer stresses that these are especially important for program analysis and verification.

11.5 Datalog+/-

Datalog+/- is described as a family of combinations of Datalog and description logics.

Motivation:

- Datalog is good at general relational structure over the active domain.
- Description logics are good at anonymous/existential individuals.
- Datalog+/- tries to combine these while preserving decidability and controlling complexity.

The lecture explains the idea of **guardedness**.

Key concept: guarded rule

A rule is guarded if all universally quantified variables occur together in a single atom in the rule body.

The lecture's rough idea:

- allow fresh variables in rule heads or function-like terms;
- but only when their use is guarded by suitable body atoms;
- guardedness helps preserve decidability.

Example guarded existential-style rule:

```
hasParent(x,y) :- Person(x)
```

corresponding to:

$$\forall x \, (Person(x) \Rightarrow \exists y \, hasParent(x, y)).$$

Here the universal variable x is guarded by `Person(x)`.

Example not guarded:

```
isRelTo(x,y) :- isRelTo(x,z), isRelTo(z,y).
```

The transitive-closure rule is not guarded in the relevant sense.

12. Datalog compared with description logic

12.1 Shared properties

Both Datalog and description logics are presented as fragments of first-order predicate logic.

Example Datalog rule:

```
Happy(x) :- hasParent(x,y), Person(x).
```

The lecture gives the corresponding first-order reading:

$$\forall x \, (Person(x) \wedge \exists y \, hasParent(x, y) \Rightarrow Happy(x)).$$

The corresponding description logic axiom is:

$$Person \sqcap \exists hasParent.\top \sqsubseteq Happy.$$

Both Datalog and the relevant description logics are:

- fragments of first-order logic;
- monotonic, in their plain forms;
- decidable;
- polynomial for the relevant reasoning tasks, with the caveat that Datalog needs bounded predicate arity;
- Horn fragments of first-order logic.

The lecture contrasts this with full first-order logic, where satisfiability/validity reasoning is undecidable.

Exam flag

Plain Datalog is monotonic, but Datalog with negation as failure is not. The bounded-arity caveat for Datalog polynomial complexity is also explicitly highlighted.

12.2 Factual vs conceptual knowledge

Both Datalog and description logic distinguish between two broad kinds of knowledge.

Factual knowledge

Examples:

```
Person(Bob)
hasParent(Bob,Sue)
```

In description logic:

- factual knowledge appears in the ABox.

In Datalog:

- factual knowledge appears as ground facts;
- the database-style term is EDB, extensional database.

The lecturer notes that factual knowledge may be huge and may change frequently.

Conceptual/class-level knowledge

Examples:

Description logic:

$$Person \sqsubseteq Mammal$$

$$Person \sqcap \exists hasChild.\top \sqsubseteq Parent$$

Datalog:

```
Mammal(x) :- Person(x).
Parent(x) :- hasChild(x,y), Person(x).
```

These are general, universally true rules that constrain models to intended ones. They are used to derive inferred facts, links, tables, or relations, and also for validation: checking whether data conforms to the intended worldview.

12.3 Expressive power: where Datalog is stronger

Higher arity

Datalog allows predicates of any arity.

Example:

```
Sale(x,y,z)
```

can directly represent a ternary relation among product, assistant, and shop.

Description logics such as $\mathcal{EL}$ and $\mathcal{EL}^{++}$ use unary and binary predicates, so this kind of ternary relation is not directly applicable.

General relational structures

Datalog can connect variables in arbitrary graph-like structures.

Example bicycle rule:

```
Bicycle(x) :- hasPart(x,y), hasPart(x,z), hasPart(x,f),
              Wheel(y), Wheel(z), Frame(f),
              connectedTo(y,f), connectedTo(z,f).
```

This expresses that the wheels and frame are connected to each other, not merely that they exist as parts. Description logic can express tree-shaped part structures, but not the same internal graph among anonymous parts.

Recursion

Datalog naturally defines recursive relations such as transitive closure.

```
isRelatedTo(x,y) :- hasParent(x,y).
isRelatedTo(x,y) :- isRelatedTo(y,x).
isRelatedTo(x,y) :- isRelatedTo(x,z), isRelatedTo(z,y).
```

The lecture presents this as a major Datalog strength.

12.4 Expressive power: where description logic is stronger

Description logics can express anonymous/existential individuals.

Example:

$$Person \sqsubseteq \exists hasParent.\top.$$

This says every person has some parent, without naming the parent.

Plain Datalog cannot express this because it cannot introduce fresh variables in rule heads.

Description logics can also express bicycle-to-parts implications such as:

$$Bicycle \sqsubseteq \exists hasPart.(Wheel \sqcap Front) \sqcap \exists hasPart.(Wheel \sqcap Back) \sqcap \exists hasPart.Frame.$$

But the lecture stresses that this gives only a tree-shaped anonymous structure, not arbitrary connections among the anonymous parts.

12.5 Complementarity summary

The lecture's main comparison:

- Both plain Datalog and $\mathcal{EL}$ lack full negation and full disjunction, though both have extensions.
- Datalog can only talk about the active domain, i.e. named individuals in the program.
- Datalog can talk about general relational structures over that active domain.
- $\mathcal{EL}$ can talk about anonymous individuals.
- $\mathcal{EL}$ can only talk about tree-shaped anonymous relational structures.
- These restrictions are chosen to keep reasoning decidable and polynomial-time.
- Datalog+/- is introduced as an attempt to combine the strengths.

12.6 Comparison table: propositional logic, description logic, Datalog

Feature	Propositional logic	$\mathcal{EL}/\mathcal{EL}^{++}$	Datalog
Predicate arity	unary-like, read p as $p(x)$	unary and binary	any arity
Conjunction	yes	yes	yes
Disjunction	yes	only limited/subclass-style	only as implication in rules
Negation	yes	no full negation; disjointness in $\mathcal{EL}^{++}$	no
Semantics	valuation, single point	interpretation with many elements	interpretation with named individuals / active domain
Main reasoning task	satisfiability	entailment, classification	entailment, classification
Complexity	NP-complete	polynomial	polynomial for low/bounded arity; ExpTime-complete if unrestricted as stated in the slide
Usage in KR	verification, HW/SW design, module for other reasoning	query answering over KBs, taxonomy design/maintenance	query answering over KBs, taxonomy design/maintenance

13. Applications of Datalog

13.1 On-stage applications: clever query language

Datalog can be used directly as a "clever" query language for data, especially in deductive databases.

Instead of writing complex SQL queries manually, one can write Datalog rules that define useful derived relations.

Example idea:

- database contains `hasParent`;
- Datalog defines `isRelatedTo`;
- user queries `isRelatedTo` rather than manually writing recursive SQL.

Datalog can be implemented:

- directly by a Datalog engine;
- by translation to SQL.

For non-recursive Datalog programs P, the lecture states that we can translate P into SQL queries Q_P^A such that:

$$P \models A(b_1, \dots, b_m) \quad \text{iff} \quad (b_1, \dots, b_m) \in Answ(Q_P^A, GrFs(P)).$$

Here $GrFs(P)$ is the extensional database consisting of the ground facts of P.

For recursive programs, translation is harder. Plain SQL does not express recursive rules such as:

```
isRelatedTo(x,y) :- isRelatedTo(x,z), isRelatedTo(z,y).
```

The lecture says recursive queries have been heavily studied in databases, and Common Table Expressions are supported by most SQL engines for general recursive queries.

13.2 Behind-the-scenes applications: program analysis and repair

Datalog is also used behind the scenes for static program analysis and repair.

In this setting, users may not write Datalog directly. Instead:

1. a tool analyses programs;
2. it translates program properties, safety conditions, and related constraints into Datalog programs and queries;
3. a Datalog engine performs the reasoning.

The slide image on page 51 depicts software engineers "on stage," with translation to a Datalog engine happening behind the scenes.

Such applications require extensions for:

- data types: reals, integers, strings;
- operations and aggregation;
- comparisons such as $\leq$ and $=$;
- complex data types: records, arrays, lists;
- pointers.

[UNCLEAR] The transcript says "lifeless conditions," which is likely garbled in the program-analysis context. Check the recording; it may be "liveness conditions."

14. Exam flags and high-value revision points

No explicit "this will be on the exam" phrase appears in the uploaded transcript, but the lecturer repeatedly flags several things as important, critical, or conceptually central.

14.1 Must-know definitions

Know the formal definitions of:

- term;
- atom;
- rule;
- program;
- fact;
- Horn clause;
- interpretation;
- substitution;
- satisfaction of atom/rule/program;
- entailment;
- Herbrand base $HB(P)$;
- Herbrand model $HM(P)$;
- immediate consequence operator ICO_P;
- monotonicity.

14.2 Very high-value theorems

Memorise and be able to use:

$$P \models \alpha \quad \text{iff} \quad \alpha \in HM(P)$$

for ground atoms α.

Also:

$$HM(P) = ICO_P^*(\emptyset)$$

and therefore:

$$P \models \alpha \quad \text{iff} \quad \alpha \in ICO_P^*(\emptyset).$$

These connect semantics, canonical models, and algorithms.

14.3 Critical complexity caveat

Datalog's Herbrand base and Herbrand model are polynomial-sized only when predicate arity is bounded. If arity is unbounded, the $p \cdot m^n$ bound can become exponential. The lecturer explicitly calls this critical.

14.4 Common mistakes to avoid

- Reading `:-` in the wrong direction. `H :- A1, A2` means $A_1 \wedge A_2 \Rightarrow H$.
- Forgetting that facts are rules with empty bodies.
- Allowing fresh variables in the head. Not allowed in plain Datalog.
- Thinking Datalog can create anonymous individuals. It cannot.
- Forgetting that plain Datalog has no negation and no disjunction.
- Computing the whole Herbrand model for a single query without recognising this may be wasteful.
- Forgetting that Datalog with negation as failure is non-monotonic.

14.5 Important conceptual contrasts

- Datalog: active domain, general relational structures, recursion.
- Description logic: anonymous individuals, tree-shaped anonymous structures.
- Plain Datalog and $\mathcal{EL}$: both restricted to keep reasoning decidable and polynomial.
- Datalog+/-: tries to combine strengths while controlling complexity.

15. Connections to earlier lectures and other material

15.1 Connection to propositional logic

Datalog rules can be translated into Horn clauses, using the same implication/disjunction equivalences seen in propositional logic:

$$\varphi \Rightarrow \psi \equiv \neg\varphi \vee \psi.$$

The Datalog rule:

```
Slippery(x) :- Rain(x), BelowF(x).
```

becomes:

$$Slippery(x) \vee \neg Rain(x) \vee \neg BelowF(x).$$

15.2 Connection to description logic

Both Datalog and description logic use model-theoretic entailment over all models, but both avoid brute-force model search using specialised reasoning methods.

Datalog's Herbrand model plays a similar practical role to canonical/consequence-based structures in earlier reasoning algorithms: it lets us compute entailments without ranging over all interpretations.

15.3 Connection to first-order logic

Datalog and description logic are fragments of first-order logic. The lecture emphasises that full first-order logic is more expressive but has undecidable reasoning tasks, whereas Datalog and description logics are designed as decidable, often polynomial fragments.

15.4 Connection to databases

Datalog is closely connected to SQL and deductive databases. Non-recursive Datalog can be translated into SQL queries; recursive Datalog relates to recursive database queries and Common Table Expressions.

15.5 Connection to program verification

Datalog can be used behind the scenes in static analysis, program verification, and repair, especially when extended with data types, comparisons, aggregation, complex structures, and pointers.

16. Unclear or transcript-garbled sections to revisit

1. **Hierarchy example: dog vs duck.** The transcript says something like "if x is a dog then x is a bird," but the slide says `Bird(x) :- Duck(x)`. The slide version is the intended hierarchy.
2. **`hasDaughter` / `hasChild` variable order.** The lecturer explicitly says the slide has a typo: both predicates should use the same variable order, likely `hasChild(x,y) :- hasDaughter(x,y)`, not one with `y,x`.
3. **Herbrand model worked example.** The displayed program includes rules deriving `Temp(W)` and `WeatherC(W)` from `Cold(W)`, but the shown Herbrand model list omits them. Check whether the slide intentionally truncated the model or whether this is an error.
4. **Model-theoretic satisfaction symbol.** The parsed slide text shows a subset symbol for atom satisfaction, but the transcript says the tuple belongs to the predicate relation. The intended condition is membership $\in$, not subset.
5. **Datalog reasoner name.** The transcript garbles the name of the disjunctive Datalog reasoner; the slide says DLV.
6. **"Brief entailment."** The transcript says "cautious entailment and brief entailment." This is likely a garbled technical term; check the recording for the exact phrase.
7. **"Lifeless conditions."** In the program-analysis application, the transcript says tools translate "program and lifeless conditions"; this is likely garbled. Check whether the lecturer said "liveness conditions."